

WORK BREAKDOWN REPORT

NDU Project

Session changes traced from the Work Breakdown Structure upwards — through Schedule, Cost Estimate and Project Controls — as raised in the Lusaka 25 (copy) planning walkthrough of 17 September 2026.

Date 23 September 2026

Branch staging

Scope 34 modified files · 26 new test files · 12 review asks traced

NAVIGATE · DELIVER · UPGRADE

Table of Contents

1. Executive Summary	3
2. The Planning Chain — How to Read This Report	3
3. Work Breakdown Structure Layer (the Base)	3
3.1 Package naming fixed at the single source	3
3.2 WBS module screen	4
4. Schedule Layer (consumes the WBS)	4
4.1 Builder screen de-cluttered	4
4.2 Schedule module screen	4
4.3 Package rows flowing in	5
5. Cost Estimate Layer (consumes the Schedule and Planning)	5
5.1 Two new pull cards — Planning data reaching the estimate	5
5.2 Technology roll-up — “270” where the table said “150/month”	5
5.3 Variance and cost descriptor text	6
6. Sections Feeding the Chain	6
6.1 Quality Management	6
6.2 SSHER	6
6.3 Design Planning	6
6.4 Stakeholder Management	6
6.5 CSV import/export round-trip	7
7. Data Model and Persistence	7
8. Project Controls (top of the chain)	7
9. Cross-Cutting Systems	7
9.1 Section flow gates	7
9.2 The shared scrollable section header	8
9.3 Tables, sidebar and performance	8
10. Traceability Matrix — Review Ask to Landed Code	8

1. Executive Summary

“Changes from the Work Breakdown Structure upwards, all the way.” — the owner’s brief for this report, 23 September 2026.

This report documents the session’s work on the NDU Project, organised the way the application itself is organised: it begins at the Work Breakdown Structure — the base of the planning chain — and follows every change upwards through the Integrated Work Package Service, the Schedule, the Cost Estimate, and finally Variance and Project Controls, closing with the cross-cutting systems that carry them all.

The session was driven by the *Lusaka 25 (copy)* voice-note walkthrough of 17 September 2026 (a roughly ninety-minute screen-by-screen review of the planning sections). Twelve of the owner’s asks were traced to landed code and tests; the full traceability matrix appears in Section 10.

Measure	Value
Modified files	34 (~4,000 lines added, ~2,700 removed)
New source files	10 (cost-line utils, gates, roll-up, shared widgets)
New test files	26
Review asks traced	12 of 12 landed
Analyzer exclusions removed	2 (cost_of_quality model + service)

2. The Planning Chain — How to Read This Report

The NDU Project guides a user through phases that build on one another. Every change in this report sits somewhere on the chain below, and most fixes were deliberately placed at the lowest point where they could be made once and inherited by every consumer above.

Layer	Module	Consumes	Feeds
Base	Work Breakdown Structure (WBS)	—	Work packages
2	Integrated Work Package Service	WBS leaves	Schedule rows
3	Schedule (Builder · Gantt · List)	WBS, packages, Cost Estimate	Purchases, dates
4	Cost Estimate	Schedule, Planning pulls	Baselines
5	Variance & Project Controls	Baseline vs actual	Forecasts, change control
Cross	Model, gates, tables, sidebar, tests	All layers	All layers

3. Work Breakdown Structure Layer (the Base)

3.1 Package naming fixed at the single source

A schedule row used to read engineering Ewp: 1.1 Authentication & Security Engineering Engineering Work Package — the classification was prefixed in raw camelCase while the title already stated its own type, and the leaf title’s last word stuttered into the appended type. The Integrated Work Package Service is now the one place that decides what a package is called on screen:

New member	Purpose
packageTypeLabel()	Human-readable classification names (EWP → “Engineering Work Package”)
packageTypeShortLabel()	Short form (“EWP”, “CWP”) used only when a title says nothing about its type
packageActivityName()	The display name: title stands as-is when it already names its type; otherwise a short prefix
packageTitleWithType()	Appends a type to a leaf title without stuttering the join
collapseRepeatedWords()	Repairs stored titles that already carry the old doubled words

All three chain-generation call sites (EWP, Procurement, Execution titles) now route through `packageTitleWithType()` instead of naive string concatenation. Upward impact: every consumer of package titles — Schedule activities and planning sync — inherits the fix automatically. Verified by the work-package activity-name test.

3.2 WBS module screen

The header stack (Section Navigator, node-count indicator, Cross-Section Sync card) moved into a shared scrollable, self-collapsing host (Section 9.2), and the section tabs were extracted to a single list shared by the navigator and the collapsed summary bar.

4. Schedule Layer (consumes the WBS)

4.1 Builder screen de-cluttered

The largest single change of the session: the Builder page lost roughly 2,100 lines of pinned, competing panels so that the Activity Tree is the page. Removed from the Builder:

- The Estimate Basis card and the Schedule Readiness Rules (pre-CWP gate checklist);
- The “Schedule Level Convention” card and its L0–L8 legend;
- The timeline (Gantt) preview — columnar and timeline views live on the List View and Gantt tabs, not here;
- The sample activity table and its footer note;
- The old camelCase name formatter, replaced by the shared package-name helpers of Section 3.1.

Guidance was reworded to point users at the row editor: *“To date an individual activity, tap its row in the Activity Tree and pick its start and finish.”*

4.2 Schedule module screen

The context banner (“Project · WBS nodes · Cost Estimate total · Planning Milestones synced · items synced”), the Resync-from-Planning button, and the scheduled-purchase pull card are preserved but now sit inside the scrollable self-collapsing header host, so the tab content keeps its share of the page on short windows.

4.3 Package rows flowing in

Planning-sync activity names now use `packageActivityName()`, so rows synced from Planning into the Schedule carry the corrected titles from Section 3.1.

5. Cost Estimate Layer (consumes the Schedule and Planning)

5.1 Two new pull cards — Planning data reaching the estimate

The owner’s asks were explicit:

“Quality is not done ... the quality costs must reach the cost estimate.”

“What is the cost aspect for these things? ... you can have them on the table and say cost items and then estimated costs ... that is how we can put our share costs into the cost estimate.”

Both are now wired, code-level, with no AI involved. Cost of Quality entries priced on the Quality tab land as **quality** cost lines (sub-category “Quality — category”, basis reference “Cost of Quality entry ...”). SSHER items ticked “**requires a purchase**” with an assessor-estimated amount land as **ssher** cost lines (unit “purchase”). Both pulls are idempotent — an existing line with the same description and total is never duplicated — and items ticked but not yet priced are surfaced as “needs a price” rather than silently dropped.

Both return typed pull results (pulled / alreadyInEstimate / addedTotal), recompute totals through the shared compute utility, and persist to storage. Selection and summing logic lives in pure, tested utils (`quality_cost_lines.dart`, `ssher_cost_lines.dart`), each with a test seam on its dashboard card so widget tests can inject data.

5.2 Technology roll-up — “270” where the table said “150/month”

The owner read **270** where the table said **150/month** and asked for “a total per table, then a subtotal for the section, then the grand total across the project duration ... and it should land as one line in the Cost Estimate.” Two real defects sat behind the complaint, and both are fixed in a new pure util rather than in the widget:

Defect	Fix
Wrong number — the old parser grabbed the first number, so “3 licences @ \$150/month” parsed as 3	<code>parseCostAmount()</code> now prefers the amount carrying a currency symbol and otherwise takes the largest number — the price in every realistic “qty @ price” form

Defect	Fix
Not a total — the card showed only one-time spend under “Total Technology Budget”, so recurring spend vanished	A full roll-up exposes one-time, recurring (per month / per year), a genuine grand total over the project’s months, and a total per table

Project duration is derived from milestone dates (any part of a month is billed), with a stated twelve-month fallback the UI labels explicitly rather than hides.

5.3 Variance and cost descriptor text

The owner asked to “*remove the double dashes, the double hyphens*” from cost descriptors. A single normalisation point now cleans descriptor text, and it is applied on both the Builder line row and the Variance screen, so the artifact cannot survive on either surface.

6. Sections Feeding the Chain

6.1 Quality Management

The Cost of Quality model and service were ported across and made live. The new `costOfQualityData` field on the shared project data model (Section 7) is the reason the Quality → Cost Estimate pipeline works end-to-end: without it, CoQ capture existed but was inert — the model field had never been carried across, which is why those two files sat excluded from the analyzer. The Quality screen also gained a tab gate notice that names which tabs are still unopened — the gate teaches the flow instead of just refusing — and a shared “consider these” hint at the top of every quality add/edit dialog, so all pop-ups suggest the same shape of thing.

6.2 SSHER

The table view is now the default (“we do need a table view ... the table view needs to be the default” — scrolling was not functional once a category held many items), with the card view one tap away because it carries the mitigation narrative. The add-item dialog gained the two fields that make the SSHER → Cost Estimate pull possible at all: **requires a purchase** and the assessor’s **estimated cost**. Purchase amounts are thousand-grouped so “12,000” reads as money.

6.3 Design Planning

Design Specifications get a read-only table as the default view — “a table cell is a poor place to write a specification,” so editing stays on the cards. Architecture Basis was made intelligible (owner: “*model 1, 2, 3 ... it is not intelligible*”): a plain-language explainer now sits above the numbered module rows, a pop-out table lets the rows be read together, and a new naming util provides proper module names.

6.4 Stakeholder Management

Three asks from the review landed on this screen:

Ask	Landing
“We need to have an actual name of the person”	A review util identifies rows that are not yet actionable (missing name, organisation, or title) so they cannot pass unnoticed
“We put a pop-up that says they need to ensure to review it”	The screen now demands review before leaving, putting the work on the user
“Where did this specific group ... you can’t just pull out random stuff”	A provenance util proves where each carried row came from (for example, carried from the preferred solution)

Related: the sidebar highlight now follows the user onto Design Planning sub-pages — the cause was exact-equality matching, replaced with prefix and alias matching in a new util.

6.5 CSV import/export round-trip

Table exports now write the same header row the template carries, so a file exported from the app can be edited in a spreadsheet and imported straight back. Row numbers are regenerated from position rather than trusted from the data.

7. Data Model and Persistence

One structural change at the model layer: the shared `ProjectDataModel` gains the `costOfQualityData` field (Cost of Quality: prevention, appraisal, internal and external failure). It is the keystone of the Quality → Cost Estimate pipeline — with it, the model and service files graduated out of the analyzer’s exclude list and into full analysis for the first time. Two consequences worth noting:

- Previously captured CoQ data, if any existed in the excluded build, becomes storable and pullable only from now on; historical projects with no stored CoQ simply show an empty card with guidance.
- The remaining excluded model files (execution quality tracking and friends) can be ported the same way when their features come up.

8. Project Controls (top of the chain)

Project Controls — the top consumer of the chain, where baseline meets actual across scope, cost, and change — received the same treatment as the modules below it: the header stack moved into the shared scrollable host, and the section tabs (Dashboard, Scope Tracking, Cost Control, Change Management, Forecasting) were extracted to a single shared list. With this, every module screen in the chain — WBS, Schedule, Cost Estimate, Project Controls — now behaves identically on short windows.

9. Cross-Cutting Systems

9.1 Section flow gates

“That design planning needs to be grayed out. So please take this action for every single section that has more than one tab ... The next must be grayed out and if you try to click on it, you should tell them to finish the flow within that section.”

One shared gate now enforces sequential completion across multi-tab sections, with messaging that names what is still unopened (see the Quality gate notice, Section 6.1). A dedicated test covers the gate rules, alongside the AI-content gate tests.

9.2 The shared scrollable section header

The root-cause fix behind the four module screens: each used to pin its header stack — the section navigator, the context banner and the status cards — above the tab content. On a short window, or as soon as one of those cards was expanded, the pinned stack squeezed the tab content instead. The new widget caps the header’s height, scrolls it, and collapses itself, so tab content always keeps its share of the page. Applied consistently to WBS, Schedule, Cost Estimate and Project Controls, with widget and module-screen scroll tests.

9.3 Tables, sidebar and performance

The launch data table gains virtualization support, with a perf smoke test and a rewritten performance README. The initiation-style sidebar wires its sub-page highlighting through the new label-matching util (Section 6.4).

10. Traceability Matrix — Review Ask to Landed Code

Every owner ask from the Lusaka 25 (copy) walkthrough that this session acted on, mapped to its landing point and its test.

#	Owner ask	Where it landed	Test
1	“Quality costs must reach the Cost Estimate”	CoQ model port (§6.1, §7) → pull + Quality card (§5.1)	quality cost lines · card e2e · tab
2	“What is the cost aspect for these things?” (SSHER)	Purchase fields on the dialog (§6.2) → pull + SSHER card (§5.1)	ssher cost lines
3	“A total per table, then a subtotal ... then the grand total”	Technology roll-up util (§5.2)	technology cost roll-up
4	“An actual name of the person” + review pop-up	Review util + screen prompt (§6.4)	stakeholder review
5	“Where did this group come from?”	Provenance util (§6.4)	stakeholder provenance
6	“Model 1, 2, 3 ... it is not intelligible”	Module labels + explainer + pop-out (§6.3)	architecture module labels
7	“Remove the double dashes”	Descriptor text util, Builder + Variance (§5.3)	cost descriptor text

#	Owner ask	Where it landed	Test
8	“The table view needs to be the default” (SSHER)	Table-default view mode (§6.2)	app table · smoke
9	“Next must be grayed out ... tell them to finish the flow”	Section flow gate (§9.1)	section flow gate · AI gates
10	“The sidebar highlight does not follow me”	Sidebar label matching (§6.4)	sidebar label match
11	Schedule rows reading “Engineering Engineering Work Package”	Package naming at the single source (§3.1)	work package activity name
12	Header stack squeezing tab content	Scrollable header across all modules (§9.2)	header scroll · widget tests

Deprioritised in the same review and logged for later, not scheduled: showing design/engineering leadership roles on the Team Management RACI display (“this is not a very high priority so can you write this down for us to come back to”).

11. Verification and Next Actions

The session’s changes are uncommitted on the staging branch: 34 modified files plus 30 untracked files (sources, tests, and the review artifacts they were built from). Twenty-six new test files accompany the changes; the three new Cost Estimate pulls are explicitly pure selection-and-summing with AI generation flags set false, in line with the project rule that cost computation is code-level, never model-generated.

Next action	Why
Run the full suite (flutter test)	26 new test files landed with the changes; green here certifies the chain end-to-end
Commit the session WIP on staging	34 modified + 30 untracked files are sitting uncommitted
Port the remaining excluded model files	Same path as the CoQ port when execution-quality features come up (§7)

Prepared 23 September 2026 · NDU Project (Navigate · Deliver · Upgrade) · Source review: Lusaka 25 (copy), 17 September 2026